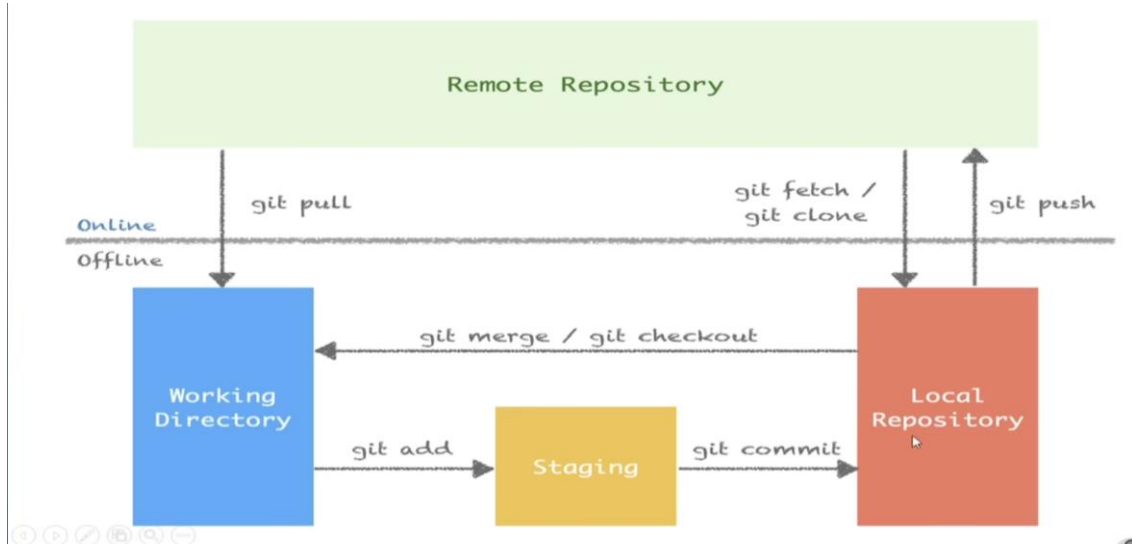


GIT/ GITHUB AND GITLAB TERMS

1.GIT WORKFLOW DIAGRAM



2.BASIC GIT AND REPOSITORY DEFINITIONS

- **Git:** Versiyonlar arası geçiş işlemlerini yapmamızı sağlayan bir araçtır.
- **GitHub & GitLab:** Yapmış olduğumuz değişiklikleri aktarmamızı sağlayan depo olarak düşünebiliriz. Yani yapmış olduğumuz değişiklikleri, dosya ve klasörleri alıp depoladığımız alanlar uzak sunuculardır (Remote Repository). Örn: GitHub, GitLab, Gitbucket.
- **Working Directory:** Çalıştığımız alan.
- **Staging Area:** Düzenlemeleri yaptığımız / hazırladığımız kısım.
- **Local Repository:** Yerel depo.
- **Repository (Repo):** Projenin ve o projeye ait bütün geçmişin saklandığı özel bir klasördür. Normal bir klasörden farkı; Git bu klasörü sürekli izler ve içindeki dosyaların geçirdiği tüm değişimleri adım adım kaydeder. İki çeşit repo vardır:
 - **Local Repository:** Bilgisayarındaki proje klasörüdür.
 - **Remote Repository:** GitHub sunucularında (İnternette) duran proje klasörüdür.

3. Uzak Sunucu (Remote) Komutları

- **git push** : Local'deki dosyayı uzaktaki sunucuya göndermeyi sağlar.
- **git pull** : Uzak sunudaki (Remote Repository) dosyayı kendi çalışma alanımıza çekmeye yarar.
- **git fetch** : Remote Repository'deki dosyayı kendi local'imize çekmeye yarar.
- **git clone** : Local'imize çektiğimiz dosyayı kaydeder / klonlar.

4. Git Terimleri ve Konfigürasyon (Git Terms)

1. git config : Git'in kimin çalıştığını bilmesi ve kişisel ayarlarını saklaması için kullanılan ayarlar dosyası komutudur.

- Kullanıcı adı tanımlama: `git config --global user.name "Zehra"`
- E-posta adresi tanımlama: `git config --global user.email "zehramugann@gmail.com"`
- Mevcut ayarları kontrol etmek için: `git config --list`

Soru / Bilgi: Buradaki "--global" bu ayarı bilgisayardaki tüm projeler için geçerli yapar. Tek tek her proje için isim vermekle uğraşmayız.

2.git init : Bilgisayarındaki normal bir klasörü Git tarafından takip edilen akıllı bir projeye (Repository / Repo) dönüştürme komutudur.

Ne yapar? :

- Sıradan bir klasörün içindeyken terminale 'git init' yazdığında:
- Git, o klasörün içine gizli bir '.git' klasörü oluşturur.
- O andan itibaren o klasördeki tüm dosya değişikliklerini izlemeye başlar.
- Kısacası klasörün 'sanal zaman makinesi' özelliğini açmış olursun.

3.git status : projenin o anki **durum fotoğrafını** çeken ve "Şu an klasörde neler oluyor?" sorusuna cevap veren en çok kullanılan kontrol komutudur. Sana projenin anlık durumunu gösteren bir **rapor** sunar.

Sana Neleri Gösterir?

Terminale git status yazdığında Git sana şu 3 temel şeyi söyler:

1. **Değiştirilmiş ama henüz kaydedilmemiş dosyalar (Modified):** Kodunu değiştirdiğin ama henüz commit yapmadığın dosyaları gösterir (Genellikle kırmızı renkle görünür).
2. **Hazırlık alanındaki dosyalar (Staged):** git add komutuyla "Ben bu değişiklikleri commit edeceğim, sıraya aldım" dediğin dosyaları gösterir (Genellikle yeşil renkle görünür).
3. **Git'in henüz takip etmediği yeni dosyalar (Untracked):** Klasöre yeni eklediğin ama Git'e henüz "bu dosyayı da izle" demediğin dosyaları gösterir.

Ne Zaman Kullanılır?

- Kod yazarken "**Hangi dosyalarda değişiklik yapmıştım?**" diye unuttuğunda,
- Commit atmadan hemen önce "**Doğru dosyaları mı kaydediyorum?**" diye kontrol etmek istediğinde terminale git status yazarsın.

4.git commit : tek başına bir komut değil; aslında **git add** ile hazırladığın değişiklikleri "zaman makinesine kalıcı olarak kaydetme" işlemidir.

1. **git add** : Kodlarında değişiklik yaptıktan veya yeni bir dosya ekledikten sonra Git bu değişiklikleri hemen kaydetmez. Önce hangi dosyaları kaydedeceğini seçmen gerekir.

- **Mantığı:** Bir kargo kutusu hazırladığını düşün. git add, kargo kutusuna koymak istediğin eşyaları seçip kutunun içine dizmektir (Buna Git dilinde **Staging Area** yani hazırlık alanı denir).
- **Kullanımı:**
 - git add dosya_adi.txt → Sadece tek bir dosyayı hazırlık alanına alır.
 - git add . (Sonundaki noktaya dikkat!) → Klasördeki **değiştirilmiş tüm dosyaları** tek seferde hazırlık alanına alır.

2. **git commit / git commit -m "message"** : git add ile dosyaları seçtikten sonra, bu anı kalıcı bir versiyon/fotoğraf haline getirmek için commit yapılır.

- **-m Parametresi Ne İş İşe Yarar?** -m, **Message (Mesaj)** kelimesinden gelir. Kaydettiğin bu versiyona ne yaptığını anlatan kısa bir not eklemeni sağlar.
- **Kullanımı:** **git commit -m "Anasayfa tasarımı güncellendi"**

3. **git rm / git rm -f** : **git rm dosya_adi.txt** → Dosyayı hem bilgisayarındaki klasörden siler hem de bu silme işlemini git add yapılmış gibi direkt hazırlık alanına ekler.

- **-f Parametresi Ne İş İşe Yarar?** -f, **Force (Zorla)** anlamına gelir.
- **git rm -f dosya_adi.txt** → Eğer bir dosyada değişiklik yaptıysan ve henüz commit atmadıysan, Git "Bak burada kaydedilmemiş değişiklikler var, emin misin?" diyerek silmeyi engeller. -f parametresi eklediğinde Git'e "*Soru sorma, uyarılara bakma, dosyayı zorla sil!*" demiş olursun.

-r

Bütün Akış Nasıl İşler? (Küçük Bir Senaryo)

1. index.html dosyasında bir buton rengi değiştirdin.
2. **git status** yazdın → Terminalde dosya **kırmızı** göründü (Değişiklik tespit edildi).
3. **git add index.html** yazdın → git status yapınca dosya **yeşil** oldu (Hazırlık alanına alındı).
4. **git commit -m "Buton rengi mavi yapıldı"** yazdın → Değişiklik mesajınla birlikte zaman makinesine **kaydedildi**.

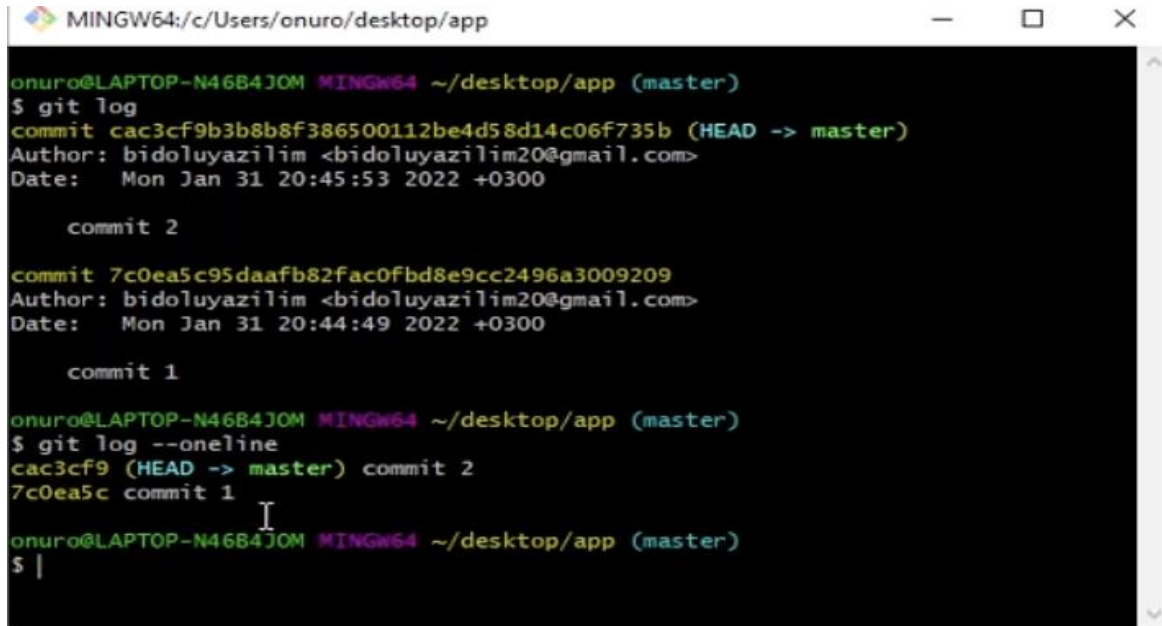
5.git log : projedeki geçmişe dönük tüm commit'lerin günlüğünü (tarihçesini) listeleyen komuttur. Zaman makinesinin "geçmiş kayıtlar" listesi gibi düşünebilirsin.

Ne İşe Yarar ve Neleri Gösterir?

Terminale git log yazdığında, projenin başından o ana kadar atılmış tüm commit'leri kronolojik olarak (en son yapılan en üstte olacak şekilde) listeler.

Her bir kayıt için sana şu bilgileri sunar:

1. **Commit Hash (Kimlik Numarası):** a1b2c3d4e5... gibi uzun, benzersiz bir harf ve sayı kombinasyonu. (Eski bir sürüme geri dönmek istediğinde bu kimliği kullanırsın.
2. **Yazar (Author):** Değişikliği kimin yaptığının adı ve e-postası (git config ile tanımladığın bilgiler).
3. **Tarih (Date):** Commit'in atıldığı tam gün ve saat.
4. **Commit Mesajı:** Değişikliği yapan kişinin yazdığı açıklama notu (-m "-" ile yazdığın kısım).



```
MINGW64: c:/Users/onuro/desktop/app
onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git log
commit cac3cf9b3b8b8f386500112be4d58d14c06f735b (HEAD -> master)
Author: bidoluyazilim <bidoluyazilim20@gmail.com>
Date: Mon Jan 31 20:45:53 2022 +0300

    commit 2

commit 7c0ea5c95daafb82fac0fbd8e9cc2496a3009209
Author: bidoluyazilim <bidoluyazilim20@gmail.com>
Date: Mon Jan 31 20:44:49 2022 +0300

    commit 1

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git log --oneline
cac3cf9 (HEAD -> master) commit 2
7c0ea5c commit 1
onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ |
```

Sık Kullanılan İpuçları 💡

git log çıktısı proje büyüdükçe çok uzayabilir. Daha temiz veya özet görmek için şu parametreler kullanılır:

- **git log --oneline** :Her commit'i tek bir satıra sığdırarak çok daha sade ve okunabilir bir özet liste verir.
- **git log -n 3** :Sadece en son yapılan 3 commit'i gösterir (sayıyı istediğin gibi değiştirebilirsin).

Terminalden Nasıl Çıkılır?

git log yazdığında liste uzunsa terminal ekranının altında iki nokta (:) çıkabilir ve klavyeden komut yazamaz hale gelebilirsin. O ekrandan/günlükten çıkıp normal terminale dönmek için klavyeden **q** (quit) tuşuna basman yeterlidir.

5.git versiyon değiştirme : projenin çalışma dizinini zaman makinesinde **geçmişteki (veya başka bir daldaki/branch'teki) belirli bir commit durumuna geri getirmek** veya projeyi o ana döndürmektir.

Bu işlem sayesinde projenin o anki kod hali tamamen değişir ve bilgisayarındaki dosyalar seçtiğin versiyonun (commit'in) tam olarak çekildiği andaki durumuna bürünür.

Ne Zaman ve Neden Yapılır?

1. **Hata / Bozulma Durumunda:** Projeye yeni bir özellik ekledin ama kodlar patladı. Tek komutla 2 saat önceki çalışan versiyona geri dönebilirsin.
2. **Eski Kodu İncelemek İstedığında:** "Geçen hafta bu sayfa nasıl çalışıyordu?" diye merak ettiğinde o güne dönüp inceleme yapabilirsin.
3. **Farklı Bir Dalda (Branch) Çalışmak İstedığında:** Ana kodları bozmadan yeni bir özellik denemek için açılan farklı dallar arasında geçiş yaparken kullanılır.

Versiyon Nasıl Değiştirilir? (Temel Komutlar)

Versiyon değiştirmek için Git'te yaygın olarak iki temel komut kullanılır :

1. git checkout (Görsel İnceleme veya Eskiye Dönüş)

Geçmişteki belirli bir commit'e geçici olarak bakmak için kullanılır:

- Önce git log --oneline yazıp dönmek istediğin commit'in kısa **hash kodunu** (örneğin a1b2c3d) kopyalarsın.
- Ardından şu komutu yazarsın: **git checkout a1b2c3d**

```
Author: bidoluyazilim <bidoluyazilim20@gmail.com>
Date: Mon Jan 31 21:10:32 2022 +0300

deneme4 eklendi.

commit 82e59a897aef7514d78365caa2bdfa3783b09150
Author: bidoluyazilim <bidoluyazilim20@gmail.com>
Date: Mon Jan 31 21:08:44 2022 +0300

deneme

commit 9729f8a516d1d703d22580d9f79318a5a4b435ea
Author: bidoluyazilim <bidoluyazilim20@gmail.com>
Date: Mon Jan 31 21:07:07 2022 +0300

deneme3 silindi

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$
onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git checkout 20edfff8382c4e47759a045cc778150150149a16 -- .|
```

(Bu durumda Git, seni "Detached HEAD" moduna sokar. Yani dosyaların o ana döner ama orada yeni kod yazıp kaydedersen ana akışa karışmaz. İncelemen bitince git checkout main veya git checkout master yazarak günümüze geri dönersin.)

6.git diff : iki farklı versiyon, commit veya dosya arasındaki **farkları satır satır gösteren** kıyaslama komutudur.Buradaki "**diff**" kelimesi İngilizce "*difference*" (fark) sözcüğünden gelir.

Ne İşe Yarar ve Neleri Gösterir?

Terminale git diff yazdığında, Git sana kodlarında tam olarak **nelerin değiştiğini** renkli bir şekilde sunar:

- **Kırmızı Satırlar (-):** Silinen veya değiştirilmek üzere kaldırılan eski kod satırlarını gösterir.
- **Yeşil Satırlar (+):** Yeni eklenen veya güncellenen yeni kod satırlarını gösterir.

```
index e69de29..25f009a 100644
--- a/deneme3.txt
+++ b/deneme3.txt
@@ -0,0 +1 @@
+deneme3
$ git diff
diff --git a/deneme3.txt b/deneme3.txt
index e69de29..25f009a 100644
--- a/deneme3.txt
+++ b/deneme3.txt
@@ -0,0 +1 @@
+deneme3
\ No newline at end of file
diff --git a/deneme4.txt b/deneme4.txt
index e69de29..b85f293 100644
--- a/deneme4.txt
+++ b/deneme4.txt
@@ -0,0 +1 @@
+sgdfhfhfgh
\ No newline at end of file
onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git diff deneme3.txt
```

En Çok Kullanılan Kullanım Şekilleri 💡

1. **git diff (Henüz git add yapılmamış değişiklikler):** Son commit'ten bu yana dosyalarda neyi değiştirdiğini, henüz hazırlık alanına (staging area) almadığın farkları gösterir.
2. **git diff deneme2.txt** : belirli bir dosyadaki değişimi gösterir.
3. **git diff --staged (veya git diff --cached):** git add ile hazırlık alanına aldığın (commit edilmeye hazır) değişikliklerin son commit ile olan farkını gösterir. Commit atmadan önce "Ben kutunun içine ne koymuştum?" diye bakmak için mükemmeldir.
4. **İki Commit Arasındaki Farkı Görmek:** git diff <commit_id_1> <commit_id_2>

Geçmişteki iki farklı versiyon arasında hangi kodların değiştiğini kıyaslar.

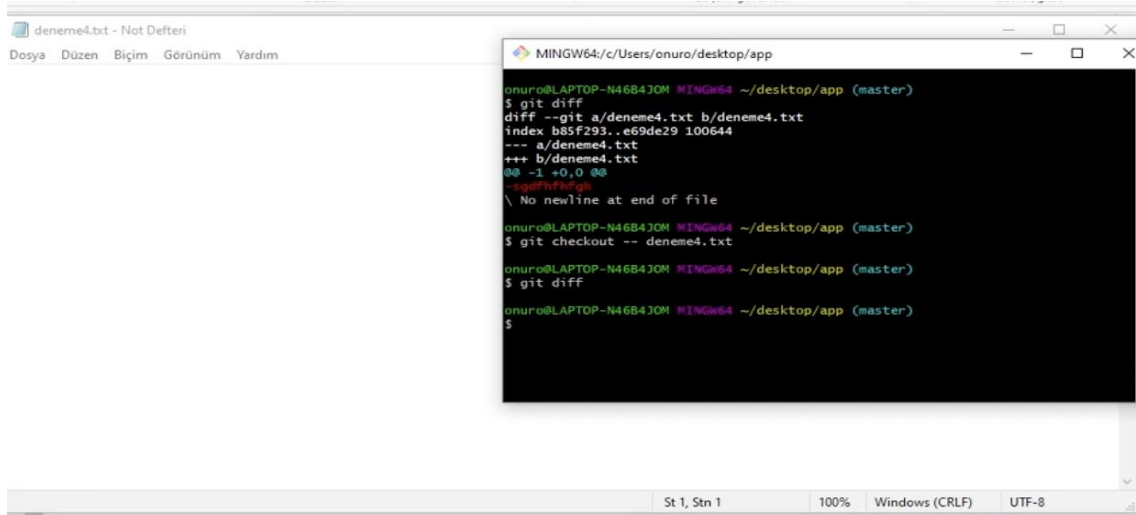
7.Git değişiklikleri geri alma (Restore) : Kod yazarken veya bir dosyayı düzenlerken elin çarptı, yanlışlıkla kodun yarısını sildin ya da tamamen yanlış şeyler yazıp kaydettin (Cmd+S yaptın). "Eyvah, kod gitti!" diye paniklemene hiç gerek yok.

İşte **git restore**, Git'in en yeni sürümlerinde bize sunulan, tam olarak bu "**Aman tanrım geri al (Ctrl+Z / Cmd+Z)**"kurtarma butonudur.

Kodunda hata yaptığında dosyanın Git içindeki **rengine** (yani nerede durduğuna) göre git restore komutunu iki şekilde kullanırsın:

1. Çalışma Alanındaysa (Dosya Kırmızıysa - git add Yapmadın)

Değişikliği yaptın, kaydettin ama henüz git add ile kutuya koymadın. git status yaptığında dosya **kırmızı** görünür.



```
onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git diff
diff --git a/deneme4.txt b/deneme4.txt
index b85f293..e69de29 100644
--- a/deneme4.txt
+++ b/deneme4.txt
@@ -1,0,0 @@
-xyzfhhfgh
\ No newline at end of file

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git checkout -- deneme4.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git diff

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$
```

- **Ne Yapacaksın?:** `git restore dosya_adi.txt` (`git restore deneme4.txt`)
- **Ne Olur?** Git, sen dosyaya hiç dokunmamışsın gibi içeriğini son çalışan kayda (en son atılan commit'e) anında geri getirir. Sildiğin yazılar pat diye geri gelir.

2. Hazırlık Alanındaysa (Dosya Yeşilse - Yanlışlıkla git add Yaptın)

Bozduğun dosyayı yanlışlıkla git add ile hazırlık alanına (Staging Area) koydun. git status yaptığında dosya **yeşil** görünür.

```
MINGW64:/c:/Users/onuro/desktop/app
diff --git a/deneme4.txt b/deneme4.txt
index b85f293..e69de29 100644
--- a/deneme4.txt
+++ b/deneme4.txt
@@ -1,0 @@
- sgdFhfgh
\ No newline at end of file

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git add deneme4.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   deneme4.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git reset HEAD deneme4.txt
Unstaged changes after reset:
M       deneme4.txt
```

Burada iki adımlı bir kurtarma yapılı:

```
modified:   deneme4.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git reset HEAD deneme4.txt
Unstaged changes after reset:
M       deneme4.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   deneme4.txt

no changes added to commit (use "git add" and/or "git commit -a")

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git checkout -- deneme4.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git diff
```

- 1. Adım: Dosyayı Yeşil Alandan (Kutudan) Çıkar: `git restore --staged dosya_adi.txt` (`git reset HEAD deneme4.txt`)
(Bu komut dosyayı silmez veya değiştireni düzeltmez; sadece yeşilden **kırmızıya** düşürür (kutudan çıkarır)).
- 2. Adım: İçeriği Eski Haline Getir: `git restore dosya_adi.txt` (`git checkout -- dosya_adi.txt`)
Şimdi birinci adımdaki gibi dosyanın içeriği tamamen ilk çalışan haline geri döner!

💡 Neden git checkout Yerine git restore Geldi?

Eski videolarda izlediğin git checkout komutu çok fazla işi aynı anda yapıyordu (hem branch değiştiriyordu hem dosya içeriğini düzeltiyordu). Git geliştiricileri kafa karışıklığını önlemek için dediler ki:

*"Bundan sonra sadece dosyaları/değişiklikleri eski haline getirmek için **git restore** komutunu kullanalım."*

Özetle: git restore, kodun patladığında veya yanlışlıkla bir şeyler sildiğinde seni son çalışan commit'ine ışınlayan en pratik güvenlik simitindir!

8.git rm : Git tarafından takip edilen bir dosyayı hem klasörden silmek hem de bu silme işlemini Git'e haber vermek için kullanılan komutudur. Buradaki **rm**, bilgisayar dünyasında *Remove* (Kaldır / Sil) kelimesinin kısaltmasıdır.

Normal Silme İle git rm Arasındaki Fark Nedir?

Proje klasöründeki bir dosyayı iki farklı şekilde silebilirsin:

1. Normal Yolla Silmek (Mac'ten Çöpe Atmak veya Sağ Tık → Delete):

- Dosyayı bilgisayardan silersin ama Git bunu *"Dosya kayboldu/değişti"* olarak algılar (git status yapınca kırmızı görünür).
- Silme işlemini Git'e kaydetmek için arkasından bir de git add dosya_adi yapman gerekir.

2. git rm İle Silmek:

- Dosyayı tek hamlede **hem bilgisayarındaki klasörden siler hem de Git'in hazırlık alanına (Staging Area) koyar** (git status yapınca direkt yeşil silindi olarak görünür).
- Yani iki adımlı işi tek adıma düşürür. Silme işlemini tamamlamak için sadece git commit yapman yeterli olur.

Sık Kullanılan Çeşitleri

1. **Dosyayı Her Yerden Silmek:** `git rm deneme4.txt`
(Dosyayı hem bilgisayarındaki klasörden hem de Git takibinden tamamen siler.)

2. **Dosyayı Sadece Git Takibinden Çıkarmak (Bilgisayarda Dursun):** `git rm --cached deneme4.txt`
(En hayat kurtarıcı seçeneklerden biridir! Dosya bilgisayarındaki klasörde kalmaya devam eder ama Git onu takip etmeyi bırakır. (Örneğin; yanlışlıkla commit ettiğin şifre dosyalarını veya özel ayar dosyalarını Git'ten çıkarmak için kullanılır).)

3. **Zorla Silmek (-f / Force):** `git rm -f deneme4.txt`
(Eğer dosyada son commit'ten sonra değişiklik yaptıysan, Git silmeni engeller ("İçinde kaydedilmemiş değişiklik var" der). -f (force) parametresi eklendiğinde Git soru sormadan dosyayı zorla siler.)

4. **Klasörü Tüm İçeriğiyle Silmek (-r / Recursive) :** `git rm -r klasor_adi`
(Bir dosya yerine tüm bir **klasörü ve içindeki her şeyi** topluca silmek istediğinde kullanılır.)

9.git mv : Git projelerinde bir dosya veya klasörün **adını değiştirmek** ya da onu **başka bir klasörün içine taşımak** için kullanılan komuttur. Buradaki **mv**, bilgisayar dünyasındaki *Move* (Taşı) kelimesinin kısaltmasıdır.

Neden Normal Yoldan İsim Değiştirmiyoruz?

Bilgisayarında bir dosyanın adını sağ tıklayıp değiştirdiğinde veya sürükleyip başka bir klasöre attığında Git bunu iki ayrı olay gibi görür:

1. "Eski isimli dosya silindi"
2. "Yeni isimli yepyeni bir dosya eklendi"

Bu durumda dosyanın geçmişteki commit geçmişi (history) kopabilir veya karışabilir.

git mv kullandığında ise: Git o dosyanın **silinmediğini, sadece adının veya yerinin değiştiğini** anlar. Böylece dosyanın geçmişteki tüm commit kayıtları ve sürüm geçmişi pürüzsüz bir şekilde korunur. Ayrıca bu işlemi direkt hazırlık alanına (git add yapılmış gibi) koyar.

Nasıl Kullanılır? (En Yaygın Senaryolar)

1. Dosyanın Adını Değiştirmek (Rename)

Bir dosyanın ismini değiştirmek istediğinde: `git mv eski_isim.txt yeni_isim.txt`
Örnek: `git mv deneme.txt anasayfa.txt`

```

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git mv deneme.txt test.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:   deneme.txt -> test.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git add .

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git commit --m "dosya ismi degistirildi."
[master cc5a234] dosya ismi degistirildi.
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename deneme.txt => test.txt (100%)

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ |

```

(Bu komut deneme.txt dosyasının adını anasayfa.txt yapar.)

2. Dosyayı Başka Bir Klasöre Taşımak (Move)

Bir dosyayı bulunduğu yerden alıp başka bir klasörün içine taşımak istediğinde: **git mv** dosya_adi.txt klasor_adi/

Örnek: git mv resim.png images/

```

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git mv test.txt dosya/
fatal: destination directory does not exist, source=test.txt, destination=dosya/

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git mv test.txt dosya/

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:   test.txt -> dosya/test.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git add .

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git commit -- "dosya tasindi"
error: pathspec 'dosya tasindi' did not match any file(s) known to git

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git commit -- "dosya tasindi"

```

(Bu komut resim.png dosyasını images klasörünün içine taşır.)

3. Hem Yerini Hem İsmi Aynı Anda Değiştirmek

: git mv eski_klasor/dosya.txt yeni_klasor/yeni_dosya.txt

```
onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git mv deneme2.txt dosya/abc.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:   deneme2.txt -> dosya/abc.txt

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git add .

onuro@LAPTOP-N46B4JOM MINGW64 ~/desktop/app (master)
$ git commit --m "dosya ismi degistirildi ve tasindi"
```

İşlem Bittiğinde Ne Yapılır?

git mv komutunu çalıştırdıktan sonra git status yaparsan, Git sana yeşil renkle **renamed: eski_isim.txt -> yeni_isim.txt** şeklinde bir rapor verir.

İşlemi tamamlamak için tek yapman gereken her zamanki gibi bir commit atmaktır:

git commit -m "Dosya adı ve yeri güncellendi"

10. git alias : Git'teki uzun veya karmaşık komutlara **kendi belirlediğin kısa takma isimleri (kısayolları)** atama özelliğidir. İngilizce "*alias*" kelimesi "**takma ad / lakap**" anlamına gelir.

- git status yazmak yerine sadece **git st**
- git checkout yazmak yerine sadece **git co**
- git commit yazmak yerine sadece **git ci**
- git branch yazmak yerine sadece **git br**

Kullanımı

```
onuro@LAPTOP-N46B4JOM MINGW64 /C/users/onuro/desktop/app (master)
$ git config --global alias.st status

onuro@LAPTOP-N46B4JOM MINGW64 /C/users/onuro/desktop/app (master)
$ git st
On branch master
nothing to commit, working tree clean
```

PULL /PUSH OPERATIONS

Push (itmek/göndermek) ve **Pull** (çekmek/almak), bilgisayarındaki yerel proje ile GitHub bulutu arasındaki veri alışverişini sağlayan iki temel komuttur.

Git Push (Bilgisayardan GitHub'a Gönderme)

Bilgisayarında kod yazıp git add ve git commit yaptıktan sonra bu değişiklikleri GitHub'a yüklemek için kullanılır.

- **İlk Kez Push Yaparken (Bağlantı Kurma):** GitHub'da boş bir repo açtığında sana bir `https://github.com/kullanici_adi/repo_adi.git` link verir (`https://github.com/kullanici_adi/repo_adi.git`). Terminalde projendeyken şu sırayla yazarsın:

`https://github.com/kullanici_adi/repo_adi.git`

`git branch -M main`

`git push -u origin main`

```
onuro@LAPTOP-N46B4JOM MINGW64 /c/users/onuro/desktop/app (master)
$ git remote add origin https://github.com/bidoluyazilim/app.git

onuro@LAPTOP-N46B4JOM MINGW64 /c/users/onuro/desktop/app (master)
$ git remote
origin

onuro@LAPTOP-N46B4JOM MINGW64 /c/users/onuro/desktop/app (master)
$ git status
On branch master
nothing to commit, working tree clean

onuro@LAPTOP-N46B4JOM MINGW64 /c/users/onuro/desktop/app (master)
$ git push -u origin master
```

(Buradaki `-u origin main`, GitHub'daki ana dal ile yerel dalını birbirine bağlar).

Sonraki Normal Push İşlemleri: İlk bağlantı kurulduktan sonra her commit attığında tek yazman gereken komut:

`git push`

Git Pull (GitHub'dan Bilgisayara Çekme)

GitHub üzerindeki en güncel kodları (ekip arkadaşının attığı commit'ler veya GitHub sitesi üzerinden yaptığın düzenlemeler) kendi bilgisayarına indirip yerel kodlarınla birleştirmek için kullanılır.

- **Kullanımı:**

git pull

(Bu komut arka planda yeni güncellemeleri indirir ve otomatik olarak senin o anki kodlarına entegre eder).

Altın Kural: Günlük Çalışma Döngüsü

1. Çalışmaya başlamadan önce güncel kodları al: git pull
2. Kodlarını yaz, değiştir ve düzenle.
3. Hazırlığa al ve kaydet: git add . → git commit -m "mesaj"
4. Güncellemeleri GitHub'a gönder: git push

.gitignore : Git'in **asla takip etmesini istemediğin** dosya veya klasörleri belirttiğin özel bir ayar dosyasıdır.

Bu dosyanın içine yazdığın hiçbir şey git status yaptığında görünmez, yanlışlıkla git add . yapsan bile Git tarafından GitHub'a gönderilmez.

Neden Kullanılır? (Hangi Dosyalar Eklenir?)

- **Gizli ve Hassas Bilgiler:** API anahtarları, veritabanı şifreleri (.env dosyaları).
- **Büyük ve Gereksiz Kütüphaneler:** Projenin indirdiği harici paketler (örneğin Node.js projelerindeki devasa node_modules/ klasörü).
- **İşletim Sistemi Çöpleri:** macOS'un otomatik oluşturduğu gizli .DS_Store dosyaları.
- **Derleme ve Çıktı Dosyaları:** C/C++ derlenince çıkan çalıştırılabilir dosyalar (.exe, .out, a.out) veya log kayıtları (.log).

Nasıl Oluşturulur ve Kullanılır?

Projenin ana klasöründeyken .gitignore adında (başında nokta olacak şekilde) bir dosya oluşturursun:

touch

.gitignore

İçine alt alta yok sayılmasını istediğin dosya veya kalıpları yazarsın:

#	Belirli	bir	dosyayı	yok	saymak	için:
sifreler.txt						
.env						

```
# Bir klasörün tamamını yok saymak için (sonuna slash koyulur):  
node_modules/  
build/
```

```
# Belirli bir uzantıya sahip TÜM dosyaları yok saymak için:  
*.log  
*.exe  
.DS_Store
```

♥ Altın Kural: Baştan Eklemek Önemlidir

.gitignore dosyasını projenin en başında oluşturup kuralları yazmak gerekir. Eğer bir dosyayı daha önce commitettiysen, sonradan adını .gitignore içine yazsan bile Git onu takip etmeye devam eder.

(Böyle bir durumda dosyayı bilgisayardan silmeden sadece takipten çıkarmak için daha önce öğrendiğin `git rm --cached dosya_adi` komutunu çalıştırman gerekir).

BRANCH KAVRAMI

Branch (Dal), ana kod düzenini ve çalışan projeyi hiç bozmadan, paralel bir evrende bağımsızca yeni özellikler denemeni veya hata düzeltmeni sağlayan çalışma hattıdır.

Bir ağacın gövdesi (main/master) ana projenizdir. Ağaçtan ayrılan yan dallar (branch) ise yeni denemeler yapıp işin bittiğinde tekrar gövdeyle birleştirdiğin bağımsız alanlardır.

Neden Branch Kullanılır?

- **Ana Kodu Korumak:** Çalışan projeyi bozma riski olmadan dilediğin gibi kod yazıp test edebilirsin.
- **Ekip Çalışması:** Aynı anda bir arkadaşın giriş ekranını kodlarken, sen profil sayfasını kendi dalında geliştirebilirsin; kodlarınız birbirine karışmaz.
- **Deneyisel Özgürlük:** Yeni bir fikir denedin ama beğenmedin mi? Ana projeye hiç dokunmadan sadece o dalı silip atabilirsin.

1. Branch Mantığı ve İş Akışı (master vs dev)

- **master / main:** Canlı sunucuda çalışan, testleri tamamen bitmiş, kararlı ve sorunsuz ana koddur.
- **dev (Development):** Yeni özelliklerin yazıldığı, hataların çözüldüğü ve testlerin yapıldığı çalışma dalıdır.

- **Temel Akış:** Geliştirmeler önce dev dalında yapılır, test edilip onaylandıktan sonra master dalına aktarılır.

2. Dalları Listeleme ve İnceleme

- **Lokaldeki Dalları Listelemek:**

- Komut: `git branch`
- Açıklama: Sadece bilgisayarında bulunan yerel dalları listeler; yanında * olan ve yeşil yanan dal o an aktif olduğunu gösterir.

```
zehra@Zehra-Dizustu-Bilgisayar robotic camp % git branch
* main
```

- **Hem Lokal Hem Uzak (GitHub) Dalları Listelemek:**

- Komut: `git branch -a`
- Açıklama: Hem bilgisayarındaki hem de GitHub üzerindeki (remotes/origin/...) tüm dalları tek seferde gösterir.

```
zehra@Zehra-Dizustu-Bilgisayar robotic camp % git branch -a
* main
remotes/origin/main
```

3. Dal Oluşturma ve Dallar Arasında Geçiş Yapma

- **Yeni Dal Oluşturmak:**

- Komut: `git branch dev`
- Açıklama: dev adında yeni bir dal açar. (Alternatif olarak GitHub web arayüzünden de branch açılabilir).

```
zehra@Zehra-Dizustu-Bilgisayar robotic camp % git branch dev
```

- **Dala Geçiş Yapmak:**

- Komut: `git checkout dev`
- Açıklama: Çalışma ortamını dev dalına taşır.

- **İzolasyon Prensipleri:** dev dalında yeni bir dosya oluşturup commit attığında, tekrar git checkout master yaparsan o dosya çalışma klasöründe görünmez. Çünkü o dosya sadece dev dalına aittir.

```
zehra@Zehra-Dizustu-Bilgisayar robotic camp % git checkout dev
Switched to branch 'dev'
zehra@Zehra-Dizustu-Bilgisayar robotic camp %
```

4. GitHub ve Lokal Arasında Senkronizasyon (fetch, push, pull)

- **Uzak Dal Bilgilerini Çekmek (fetch):**

- Komut: `git fetch`
- Açıklama: GitHub'da yeni açılan bir dalın bilgilerini yerel bilgisayara tanıtır.

```
zehra@Zehra-Dizustu-Bilgisayar robotic camp % git fetch
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 935 bytes | 311.00 KiB/s, done.
From https://github.com/zehramugan/robotic-camp
 0ca7b5f..c7c26d5  main    -> origin/main
 * [new branch]   dev      -> origin/dev
```

- **Belirli Bir Dala Push Atmak:**

- Komut: `git push -u origin dev`

- Açıklama: dev dalındaki yeni commit'leri doğrudan GitHub'daki dev dalına yükler.
- **Belirli Bir Daldan Güncellemeleri Çekmek:**
 - Komut: `git pull origin dev`
 - Açıklama: Başka bir yazılımcının dev dalına gönderdiği yeni değişiklikleri kendi bilgisayarına indirir.

5. Dalları Birleştirme (git merge)

dev dalındaki değişiklikleri ana master dalına katmak için uygulanan adımlar:

1. Önce ana dala geçilir: `git checkout master`
2. Kodlar birleştirilir: `git merge dev`
3. Birleşen yeni kodlar GitHub'a gönderilir: `git push origin master`

(İsteyenlerin bu birleştirmeyi GitHub üzerinden **Pull Request** açarak da yapabileceği belirtilir).

6. Dal Silme ve Temizleme İşlemleri

- **Lokaldeki Dalı Silmek:**
 - Komut: `git branch -d test`
 - Açıklama: İşi biten yerel test dalını bilgisayardan siler.
- **Uzakta Silinen Dalları Yerelden Temizlemek (prune):**
 - Komut: `git fetch -p` (veya `git fetch --prune`)
 - Açıklama: GitHub üzerinden silinmiş ama lokalde hâlâ izi kalmış dalların referanslarını temizler.